

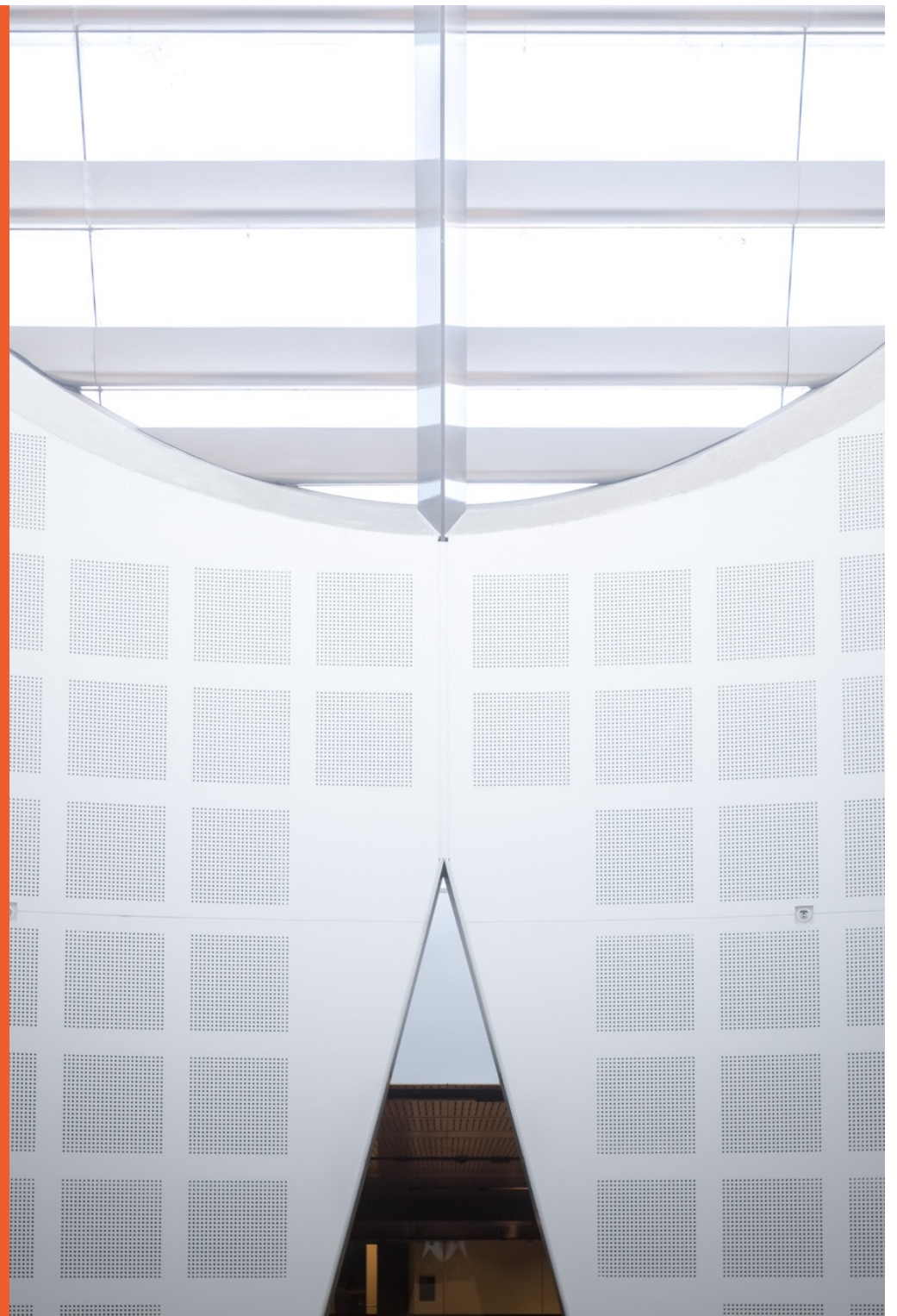
Front-end Development Building for the browser

Presented by
Dr. Huaming Chen

ELEC3609/9609
Internet Software Platforms
2025 S2



THE UNIVERSITY OF
SYDNEY



We acknowledge the tradition of
custodianship and law of the Country on which
the University of Sydney campuses stand.
We pay our respects to those who have cared
and continue to care for Country.



THE UNIVERSITY OF
SYDNEY

Overview

- ✓ **User Experience**
- ✓ **Webpage Design**
- ✓ **The Browser**
- ✓ **HTML**
- ✓ **CSS**
- ✓ **JavaScript**

Introduction

- The frontend is the presentation layer.
- It serves as the interface between the user and the backend (data access, computations, etc).
- This lecture deals with the software components that execute in the **browser**.
 - *“Mobile is a magnifying glass for your usability problems.”*

A top-down view of a design workspace. In the center, a pair of hands is using an orange-handled utility knife to cut a piece of orange paper on a green cutting mat. The mat has white grid lines and measurements. To the left of the hands, a clear plastic ruler lies diagonally. Above the hands, there are several pieces of colored paper: a yellow one, a light blue one, and a pink one. To the right of the hands, a wooden bone folder is lying horizontally. The background is a plain, light-colored surface.

**Design is not art. It is
about crafting solutions
to real issues.**

Mark Boulton

Good User Experience

- Building effective user interfaces is not art.
- Effective user interfaces are **sensible, consistent** and **structured**.
- If you can develop software, you can build an effective user interface (if you dedicate the time to it).

User Experience vs User Interface

For UX designer:

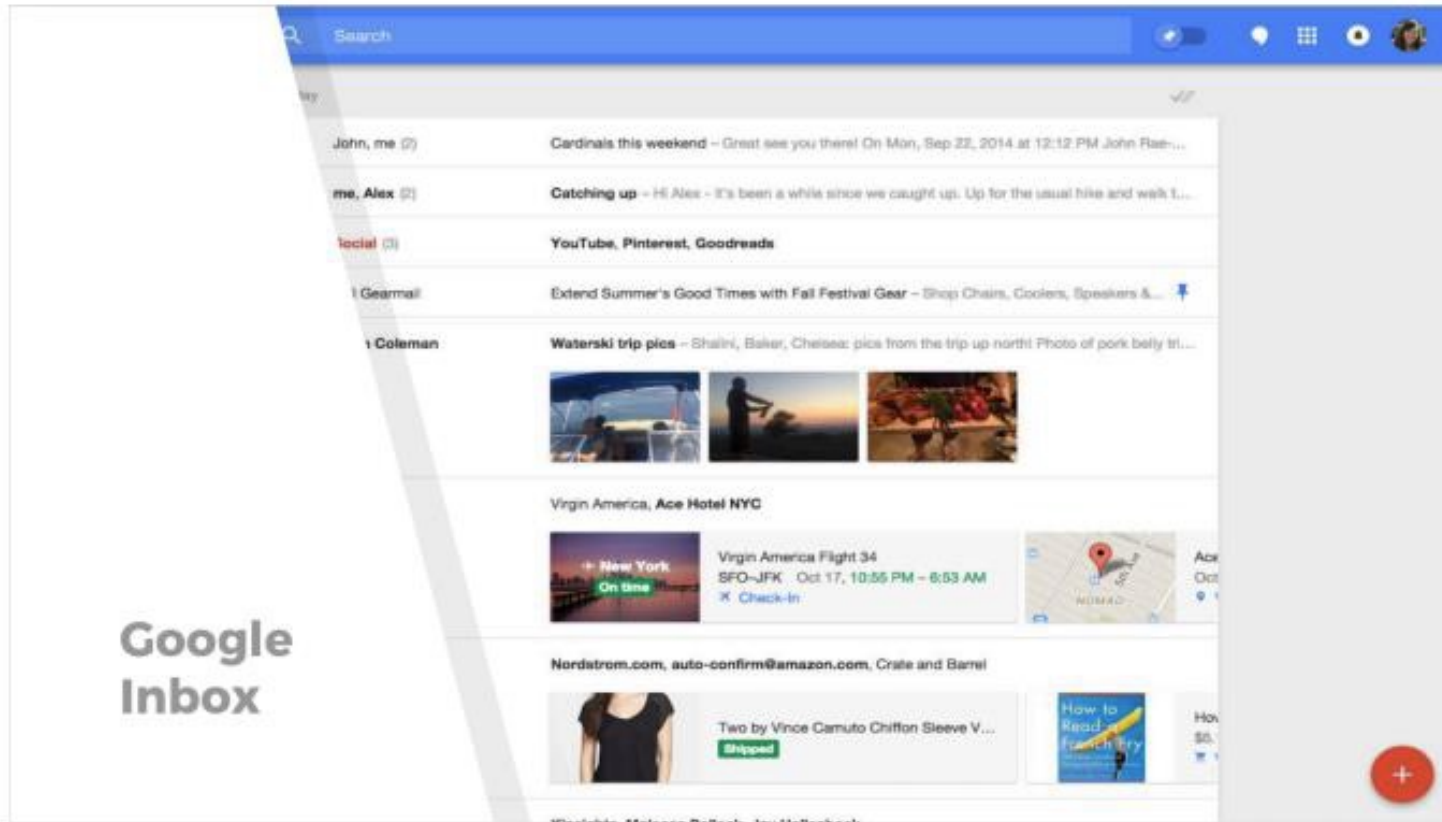
- Interaction designer
- Charts the user pathway
- Plans information architecture
- Expert in wireframes, prototypes, and research

For UI designer:

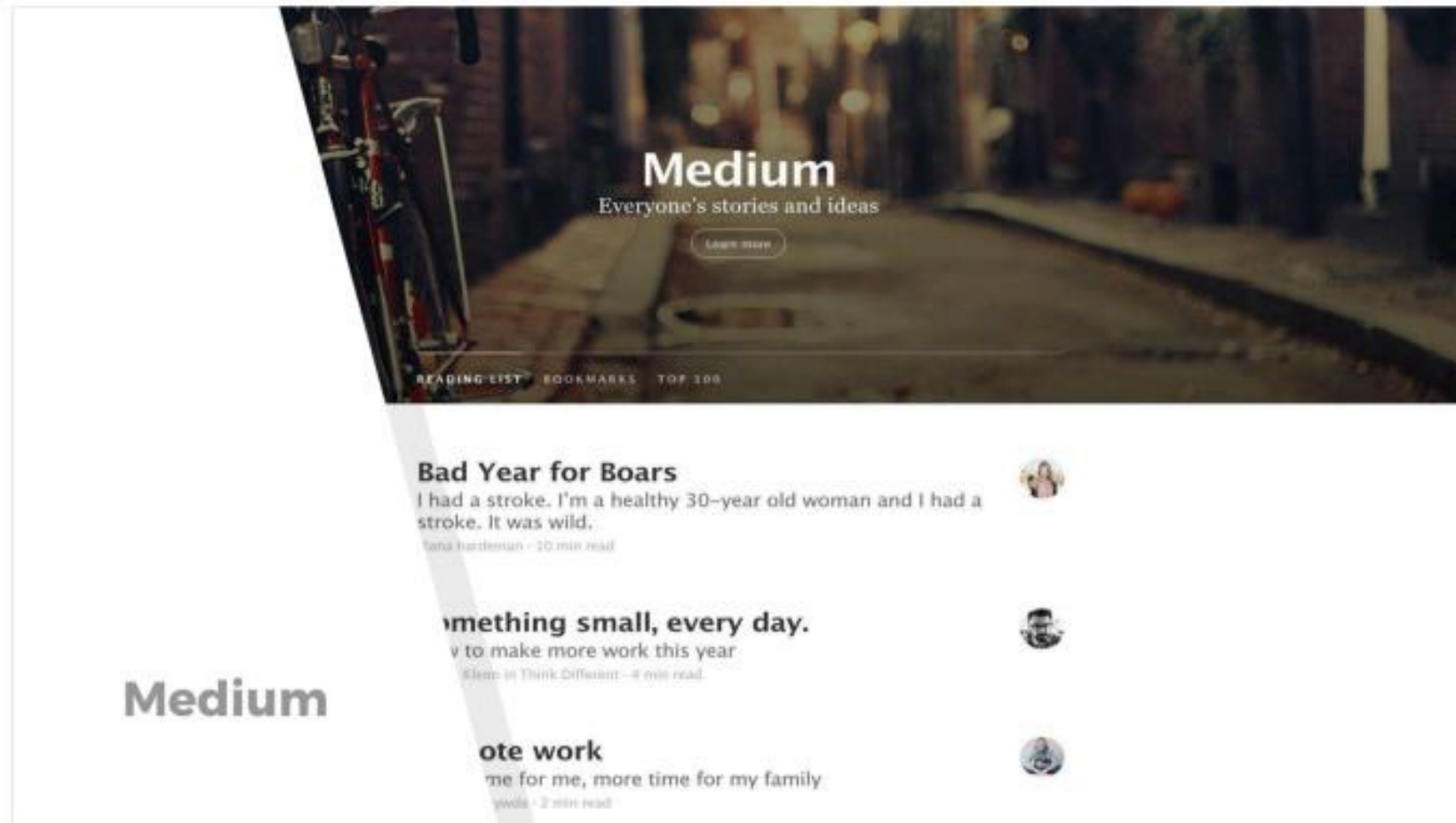
- Visual designer
- Chooses color and typography
- Plans visual aesthetic
- Expert in mockups, graphics, and layouts

Technical Artists?

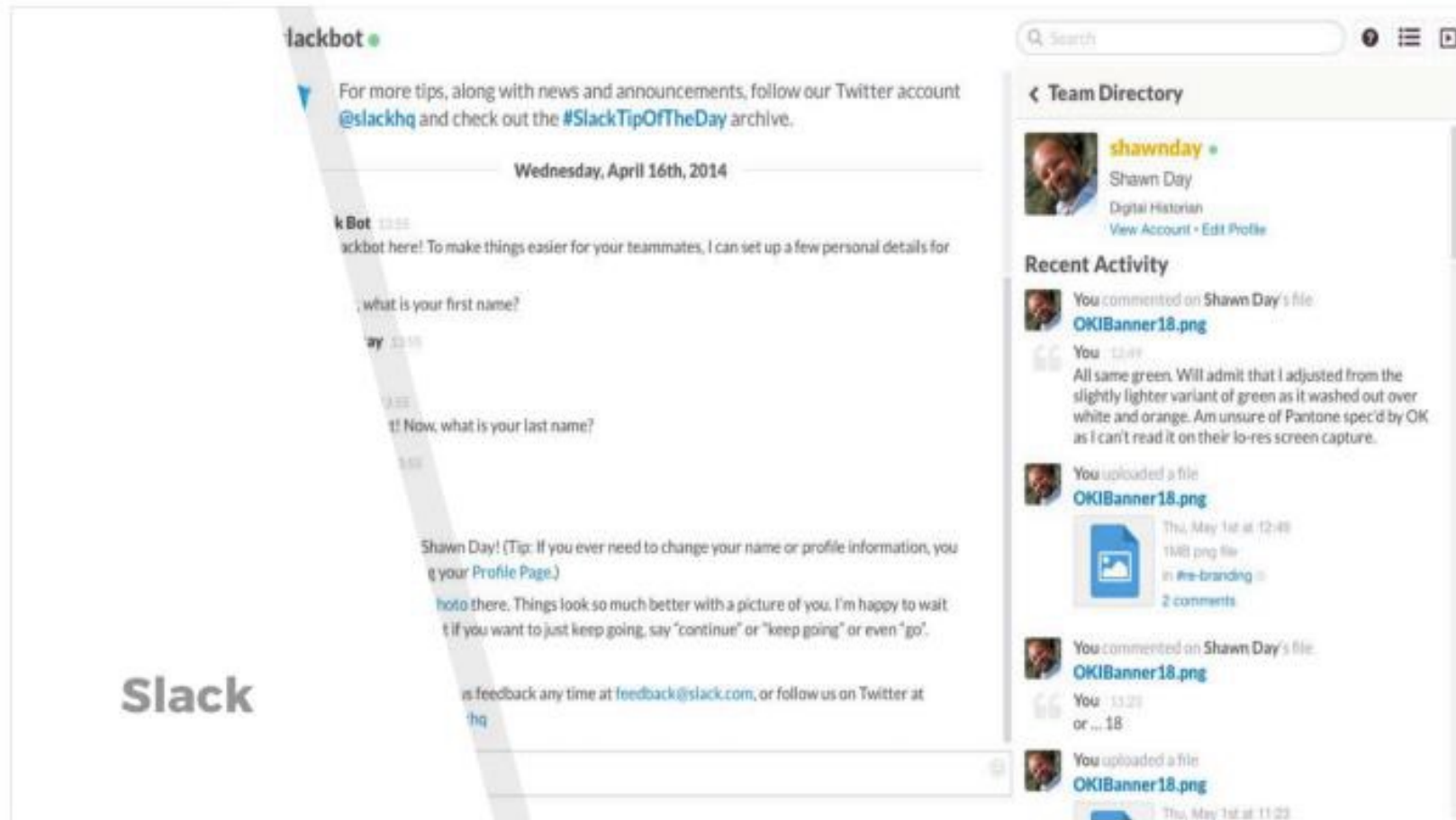
Google Inbox



Medium



Slack



Good User Experience

- Users shouldn't need instructions.
- Simple steps to achieve common tasks.
- Every click matters.
- Make the user happy!

*“Most people make the mistake of thinking design is what it looks like. People think it’s this veneer – that the designers are handed this box and told, ‘Make it look good!’ That’s not what we think design is. **It’s not just what it looks like and feels like. Design is how it works.**”*

- Steve Jobs

It's not you. Bad doors are everywhere.



LingsCars

I live inside my car leasing website all day Monday to Friday 9am-6pm. I lease the cheapest PCP and contract hire cars in the UK! - I am Ling, accept no substitutes

LING'sCARS.com

Leader of the Pack - The UK's favorite car leasing website!
Contract hire cars from Ling Valentine. LING'sCARS is the UK's favourite car leasing website - On 2016 I leased over £85 million in cars! (RRP)

CLICK HERE TO CHAT LIVE

I AM LING YOU CAN TRUST ME

Menu

- Home
- Cars
- Vans
- Customers
- About Ling
- Fun stuff
- Free stuff
- Live staff

CARS A-Z

13 DELIVERIES IN LAST WEEK

Seat Leon	Chris	CAMBRIDGE, CB24
Ford Kuga Estate	Matt	FELTHAM, TW13
Skoda Octavia Estate...		NEWTON ABBOT, TQ12
Skoda Octavia Estate...	Paul	NEWTON ABBOT, TQ12
Skoda Octavia Estate...	Andrew	SEDGEFIELD, TS21
BMW 2 Series Coupe	Paul	HERTHYR TYDFIL, CF48
Citroen C3	Paul	WALSALL, W55
Mazda 6 Saloon	Gary	LONDON, N13
Renault Captur		

BREXIT RISK FREE!

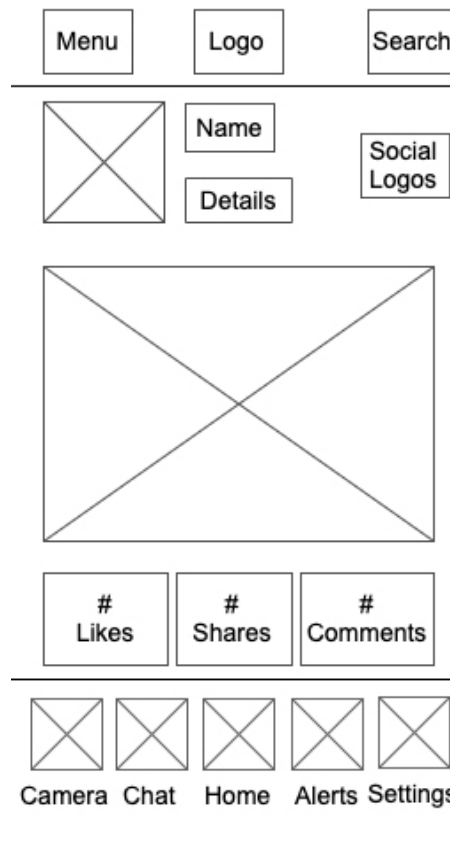
LING's BREXIT BORIS PROMISE: If the UK leaves the EU with "no deal", tariffs may apply to EU built cars landing in the UK after Brexit day (31st Oct?). If this is the case with YOUR car, and IF the price increases due to tariffs, YOU will be able to cancel the car order at **NO COST** to you. **YOU can order a new car from me, with NO BREXIT RISK.** - Ling

MY BEST SELLING

Webpage Design

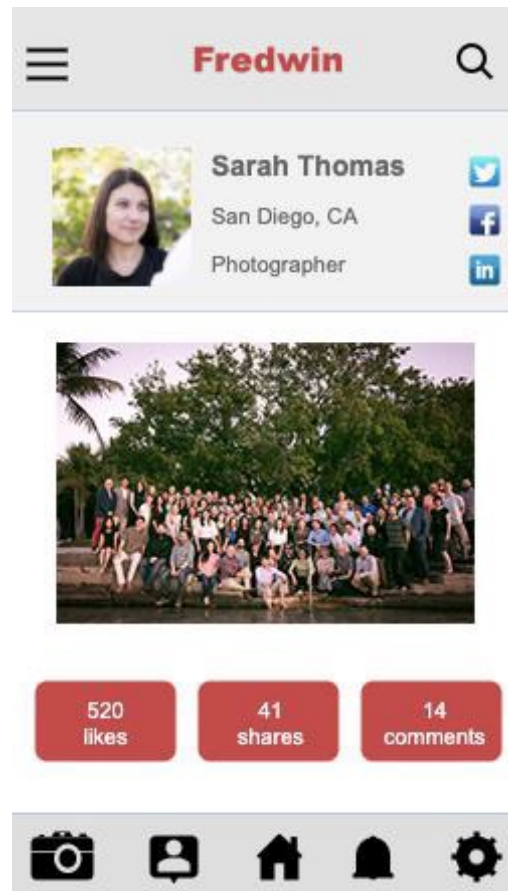
Wireframe

- Shows the information that will be displayed.
- An outline of the structure and layout of components.
- A blueprint for each page that will be implemented.



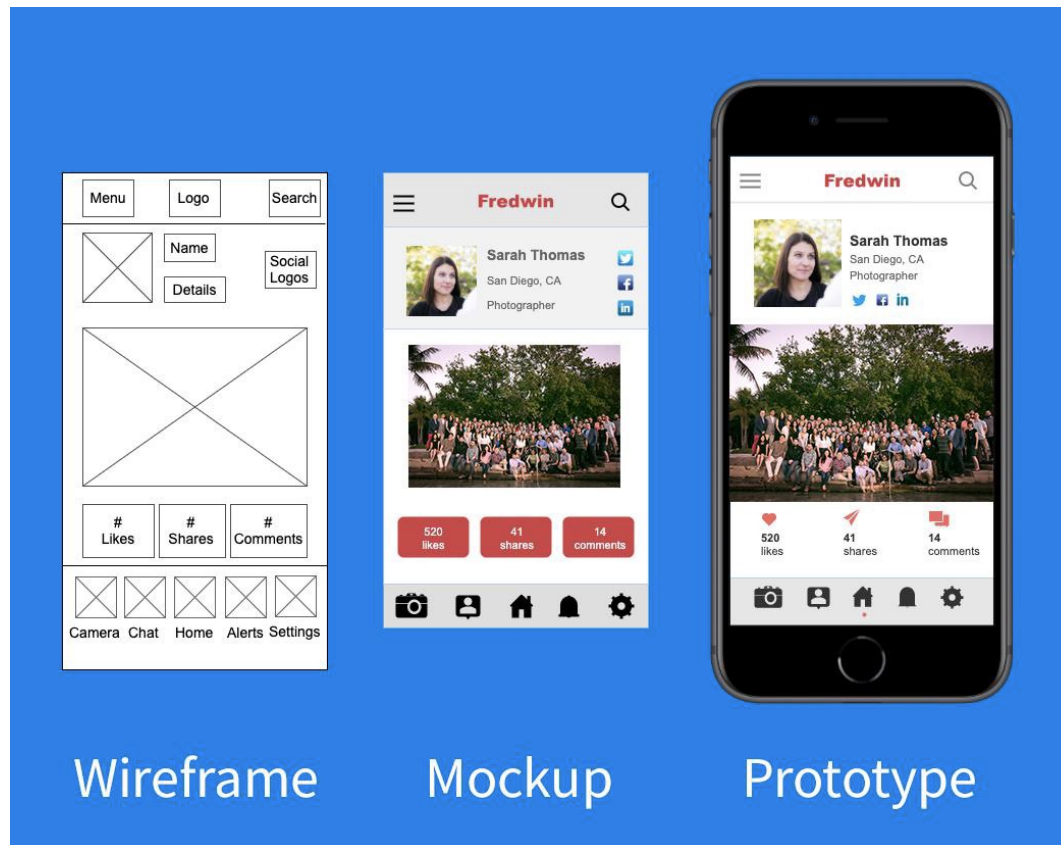
Mockup

- **Key information:** Visual appearance of the page.
- Should be more detailed than a wireframe.



Prototype

- **Key information:** How the page works
- Can be low or high-fidelity focus is on how it behaves.
- Users should be able to use and interact with your prototype.



The Browser

Not All Browsers Are Created Equal

Each browser is different

...and so is each version of a browser

...and each version is different on each OS

...don't forget different screen resolutions

...or mobile devices

- You must consider interoperability between browsers, browser versions and operating systems when writing web applications.

Can I Use?

caniuse.com provides a quick reference on browser compatibility for features and limitations.

Cross-Compatibility Testing

- If our application must run on a certain browser, we must test it on that browser. **Never assume it will work.**
- We can test different browsers by using our own local machine.
- We can test different operating systems by emulating them with virtual machines. Virtualisation can introduce bugs itself.

Professional Solutions

Many professional tools and services exist for testing cross-compatibility, often with the ability to test on real hardware and devices (e.g., Windows Phone).

You Don't Need To Support ALL Browsers!

Strategically choose your supported browsers.

- You don't need to (and probably shouldn't) support all browsers, versions and operating systems.
- At the start of your project, make a decision on what you will and won't support. Base this decision on your intended customer base.
- Know your intended audience.

Design practice

Design practice

- Responsive design
- Inclusive design
 - Design that does not exclude on the basis of
 - Disability, religion, gender identity
 - Race, language, sexual orientation
 - Ethnicity, pronunciation, age



Resources

- <https://ui-patterns.com/patterns>
- https://hike.one/?utm_source=androidpatterns.com/
- <https://www.uxlibrary.org/explore/ui-design/ui-patterns-and-inspiration>

Web Component

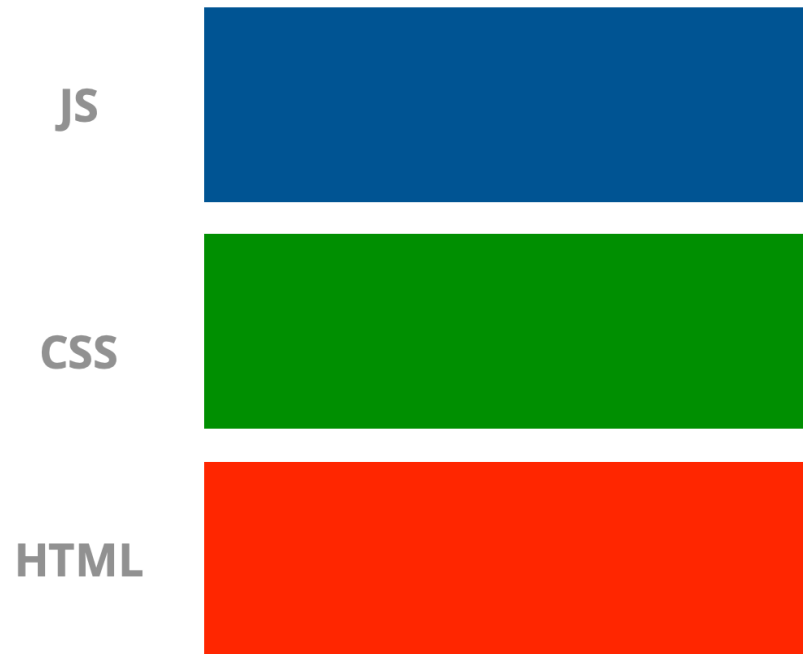
Web Component

- Custom HTML elements and technologies that facilitate their development
- Components on the Web
- Any reusable blob of HTML, CSS, and JS

Web Component

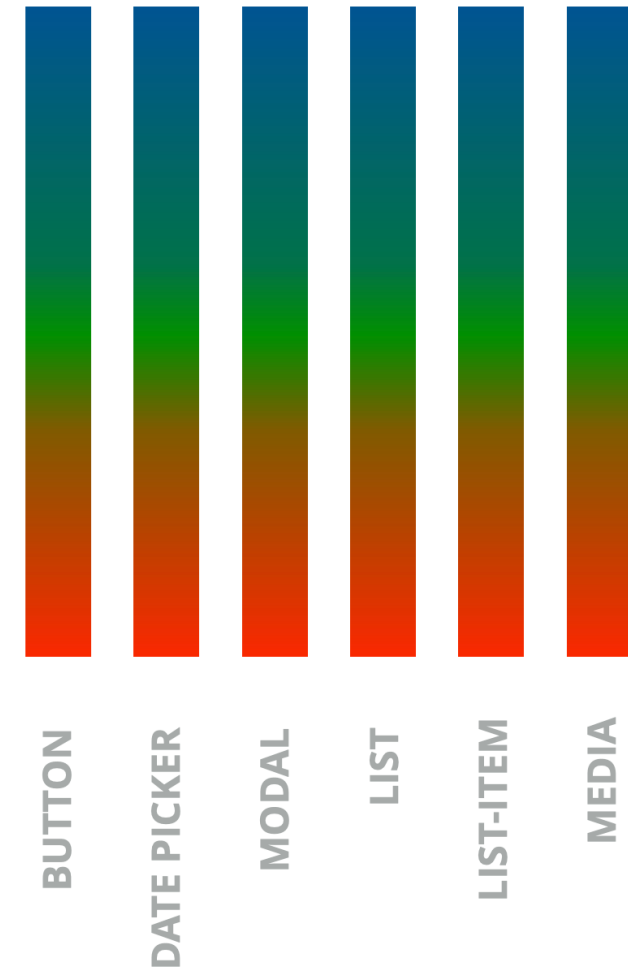
- Reactivity
- Encapsulation
- Modularity
- Reusability

Separation of Concerns



Separation of Concerns

(only, from a different point of view)



Sources from internet

The University of Sydney

HTML Markup

HTML Versions

HTML is a constantly evolving specification.

Summarised Timeline:

- 1997: HTML 3.2
- 1999: HTML 4.01
- 2007: HTML5 Draft Published
- 2014: HTML5 Published as W3C Recommendation

Browsers implement subsets of the HTML specifications.

- `<audio>` `<video>` elements
- ``data``, ``time``, ``color`` input types in HTML5

A Minimal HTML5 Document

- A simple HTML5 document implementing CSS and JavaScript.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>MINI HTML5</title>
  <link rel="stylesheet" type="text/css" href="style.css">
  <script src="script.js"></script>
</head>

<body>
  <header>
    <h1>This is a Heading!</h1>
  </header>
  <article>
    <p>This is a paragraph.</p>
  </article>
</body>
</html>
```



DOCTYPE Tag

`<!DOCTYPE html>`

- Not a HTML tag. It is an instruction to the browser of the version of HTML used by the page.
- Must be the **very first** line in your document.
- This is a HTML5 DOCTYPE declaration, however it will masquerade as HTML4 for older browsers.
- HTML 4 DOCTYPES are more complex.

- `<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">`

- `<!DOCTYPE html>`

Element

<html>

<head>

<title>

<body>

<h1>

<p>

<tagname> Content goes here... </tagname>

HTML Tag

```
<html>  
  ...  
</html>
```

- Encapsulates the entire HTML document.
- Can specify the language used in the document.

```
<html lang="en">
```

HEAD Tag

```
<head>
  <meta charset="UTF-8">
  <title>MINI HTML5</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js"></script>
</head>
```

- Contains meta information that is separate from the content.
- Elements are not rendered in the view.
- CSS files are generally imported here.
- JS imported here will delay rendering of the page until the scripts are loaded. JS can instead be imported in the *body* to speed up page loading times.
- Title tag is mandatory in this section.

BODY Tag

```
<body>
  <h1>This is a Heading!</h1>
  <p>This is a paragraph.</p>
</body>
```

- Contains the HTML elements to be rendered to the view.
- If placing script tags in the section, it is convention to place them just before the closing body tag.
- Body is a mandatory tag for a HTML document.
- Never Skip the End Tag
- Empty elements:

Attributes

<a>

<p>

...



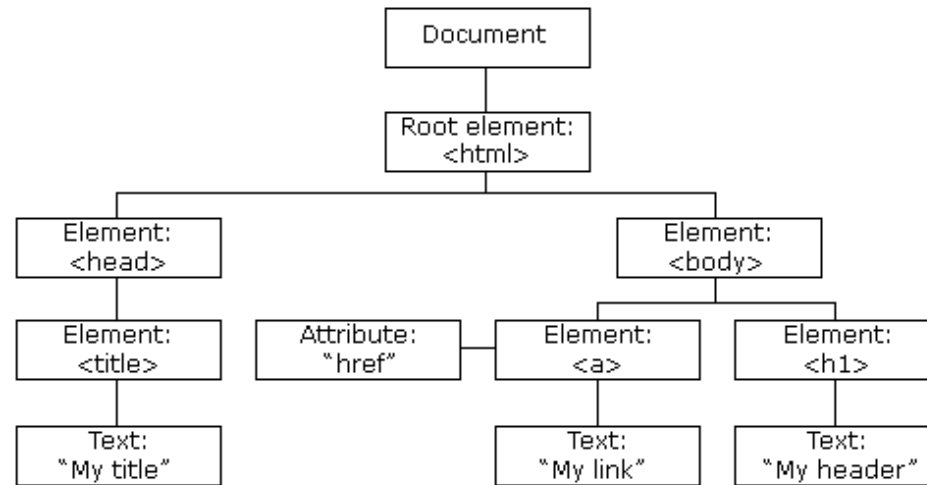
Visit
USYD

<p style="color:red;" title="I'm a
tooltip">This is a red paragraph.</p>

...

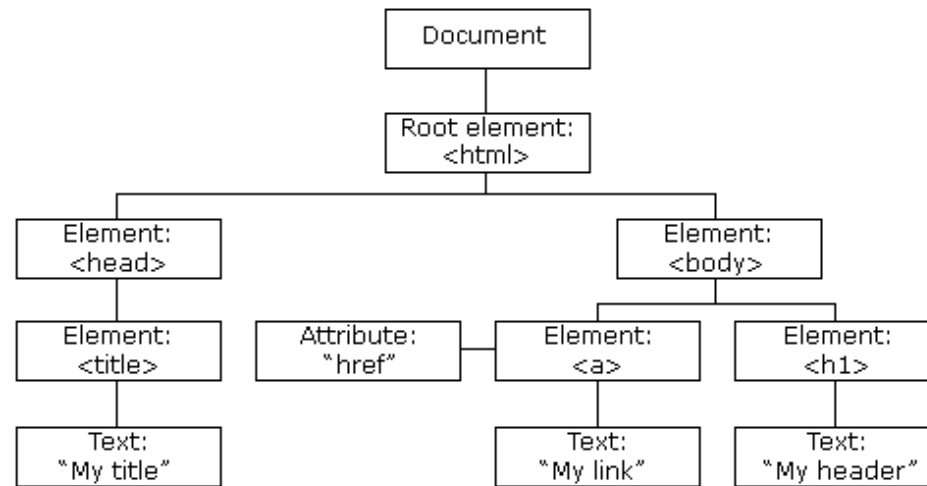
- Always Use Lowercase Attributes
- Always Quote Attribute Values
- Single or Double Quotes?
-

The DOM



- The document object model treats a HTML document as a tree-like structure of objects.
- Manipulating the DOM is possible from JavaScript, and forms the majority of JavaScript usage in the browser.

The DOM



- Finding Elements by
 - Id (getElementById), Tag name (getElementsByTagName)
 - Class Name (getElementsByClassName)
 - ...

An HTML Custom element consists of...

- Changing HTML content
- Updating values of an attribute
- Dynamic HTML content (i.e., Date())
- Write directly to HTML output stream
- ...

A JS class

+

A tag name

```
class MyElement extends HTMLElement {  
    constructor() {  
        super();  
    }  
}  
  
// Register <my-element>  
customElements.define("my-element", MyElement);
```

The DOM

- What gets logged?

```
document.createElement('my-element');
```

- a. 'connected', 'disconnected'
- b. Nothing
- c. 'disconnected'
- d. 'connected'

```
// Define a custom element
class MyElement extends HTMLElement {
  connectedCallback() {
    console.log('connected');
  }

  disconnectedCallback() {
    console.log('disconnected');
  }
}

// Register the custom element
customElements.define('my-element', MyElement);

// Create the element
const element = document.createElement('my-element');

// Append the element to the DOM (triggers connectedCallback)
document.body.appendChild(element);

// Remove the element from the DOM (triggers disconnectedCallback)
document.body.removeChild(element);
```

The DOM

- What gets logged?

```
let a = document.createElement("div");  
let b = document.createElement("my-element");  
a.append(b);
```

- a. 'connected', 'disconnected'
- b. Nothing
- c. 'disconnected'
- d. 'connected'

```
// Append 'a' to the DOM (this will trigger connectedCallback for 'b')  
document.body.appendChild(a);
```

CSS

Cascading Style Sheets

- Describes the presentation of a HTML document.
- Enables a separation of concerns between “what’s in a document” (HTML) and “what a document looks like” (CSS).
- Similar to HTML, CSS has different versions and varying support across browsers.
- CSS can be difficult to get right but is 100% worth it.

CSS Syntax

selector { property:value; }

Selector: Points to the HTML element the style should apply to

Property: The attribute that should be modified.

Value: What the value should be updated to.

{}: Declaration block can contain multiple property value pairs.

```
selector {  
  property:value;  
  property:value;  
}
```

Selectors: Element

```
body {  
    background: #eeeeee;  
    font-family: Arial, Helvetica, sans-serif;  
}
```

- Selects the HTML element *body*, applies a background color and defines a font to be used for text.

Selectors: Class

```
.dog {  
    color: blue;  
    font-weight: bold;  
}
```

- Any elements with the *dog* class will have bolded, blue font.
- A class may be applied to multiple elements. Elements may have multiple classes.

Selectors: ID

```
#dog-name {  
    color: red;  
    text-decoration: underline;  
}
```

- The first element in the HTML page with the *dog-name* id is now underlined and has red font.
- An id is used to select **only one** element. An element in a HTML document may have **only one** id.

Media Queries

```
@media only screen and (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}  
  
@media only screen and (max-width: 400px) {  
  body {  
    background-color: lightgreen;  
  }  
}
```

- Media queries allow us to customise how our document displays on a range of devices by applying conditions to CSS rules.
- Applies CSS if the @media query returns true.
- Great for creating responsive pages.

Best CSS?

The best CSS is ...

CSS Frameworks

**The best CSS is written by someone else.
Use a framework.**

Choosing a CSS Framework

- Popular frameworks:
 - [Bootstrap](#)
 - [Foundation](#)
 - [Materialize](#)
- **Choose your framework carefully.** Switching between frameworks can be laborious and time-consuming.
- When choosing a framework, consider:
 - Aesthetics
 - Grid system
 - Community support
 - Cross-browser compatibility

```
<div class="container">  
  <div class="row">  
    <div class="col-sm-6">Column 1</div>  
    <div class="col-sm-6">Column 2</div>  
  </div>  
</div>
```

JavaScript

JavaScript

- JavaScript is a **high-level, dynamic, un-typed, interpreted** programming language.
- JavaScript allows pages to become dynamic by manipulating the HTML DOM with code.
- Alongside HTML and CSS, it forms a fundamental core of today's modern web.
- Learning to appreciate JavaScript will improve your applications and make you more employable.

```
<p id="message">Hello, World!</p>
<button onclick="changeText()">Change Text</button>

<script>
  function changeText() {
    document.getElementById("message").textContent = "New Text!";
  }
</script>
```

JavaScript Issues

- Early versions were hard to develop in:
 - Weird equality/automatic conversion.
 - Behaviour of *this*.
 - Lack of a type system.
 - Weak static analysis support. Poor tooling.
 - Lack of a module system.
- JavaScript is constantly evolving, and the language has improved significantly with more recent versions.

JavaScript isn't perfect but it's here to stay.

TypeScript

- TypeScript is an open-source programming language that implements a superset of JavaScript while adding optional static types.
- Developed and maintained by Microsoft and designed for the development of large web applications.
- Transpiled back to JavaScript

Adds support for:

- types
- modules/namespaces
- classes

JQuery

JQuery – the Swiss army knife!

- Commonly used on web pages.
- Highly supported and has an active community.
- Likely to have everything you need to interact with the DOM.
- Great for cross platform support.

Becoming less popular due to modern frameworks superseding JQuery's functionality and browsers implementing native versions of a lot of the library.

Single Page Applications

- Single page applications load a single HTML document and use JS to update the user's view without a hard refresh.
- Single page applications move a lot of work to the client-side (browser) and typically create a smoother, more fluid user experience. They allow for significantly more powerful web applications.

Single Page Applications

Pros:

- Provide a faster interface.
- Encourage separation of concerns.
- Reusable
- Allow for “serverless” browser applications.
- Simpler binding of data to views.
- Easy to serve from a CDN, decreasing user load time.
- Removes load from backend servers.

Cons:

- Typically require more work.
- Typically require more experienced developers due to complexity.
- New frontier. Frameworks are rapidly evolving.
- Pushes workload into the browser, bad for low-spec/mobile devices.

AJAX

Asynchronous JavaScript and XML

- Allows the page to be updated with external data without a full refresh.
- AJAX is a fundamental aspect of the modern web.
- Traditionally we would use XHR, which can be clunky and cumbersome.
- JavaScript libraries like JQuery improved the situation
- Native browser APIs, such as Fetch, are also an option where supported.

Popular JavaScript Frameworks

- Angular (Google)
- React (Facebook)
- Ember
- Backbone.js

Development Tools

Bower

Bower – a package manager for the web.

- Imagine **pip** but for frontend HTML, JS and CSS libraries.
- A Node.js application – installed via **npm**.
- Can fetch packages and their dependencies.
- Much easier to update packages.
- Document your dependencies in a *bower.json* file.



Grunt

Grunt – a JavaScript task runner.

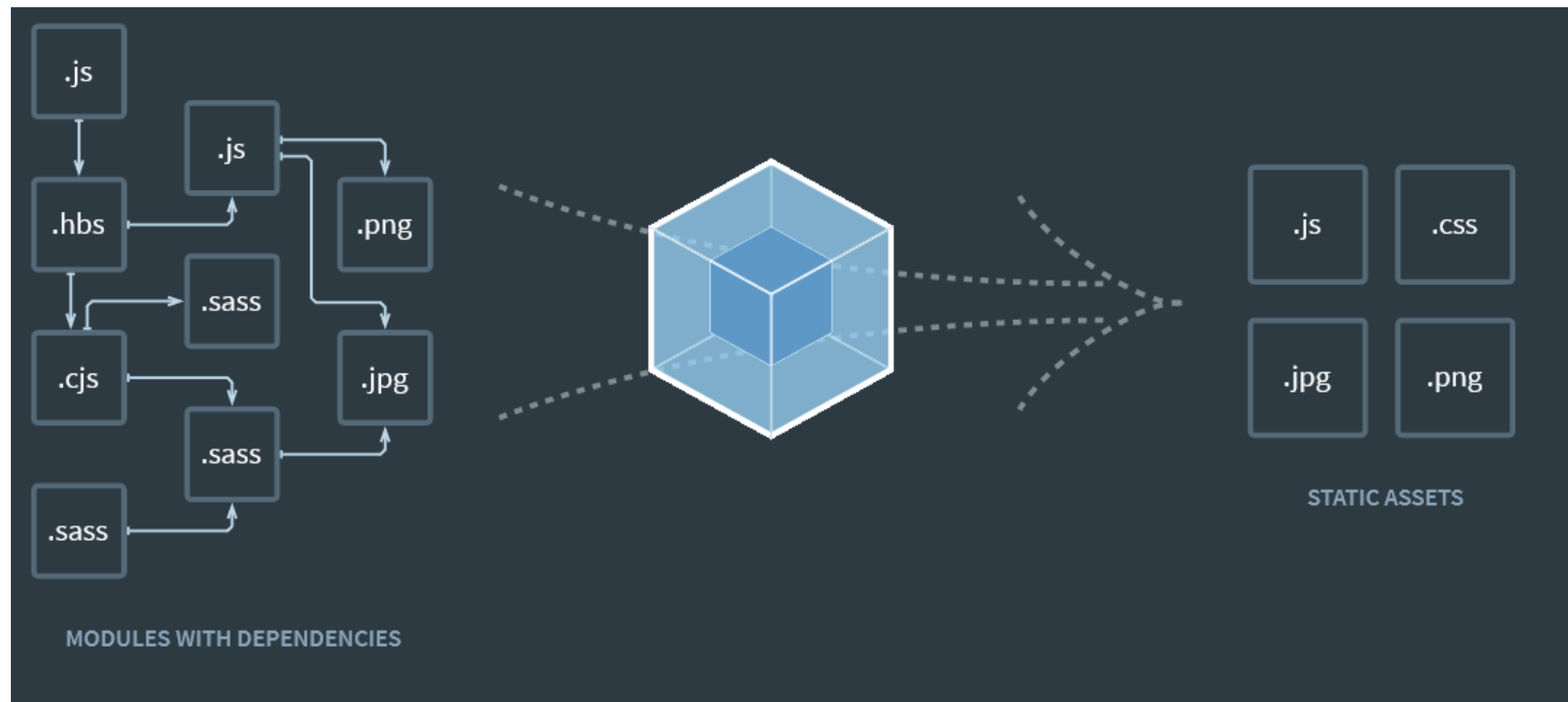
- Similar to a Makefile but for HTML frontends.
- Also a Node.js application – installed via **npm**.
- Automates boring, repetitive tasks:
 - minification of images, CSS, JS, HTML
 - dependency linking
 - linting (error/style checking)
 - testing

Gulp is another popular JS task runner.



Webpack

- JavaScript module bundler.
- Generates static assets with only the dependencies needed.



**You reached the end of the lecture.
Thank you and see you next week.**